

Introduction

Every large codebase has a version of this problem hiding in it. A function gets flagged as broken, and the real question is never "how do I delete this one function" — it's "how much of the system does this function's blast radius actually cover, and is any of that blast radius something other code still relies on." That second half is the part people skip, and it's exactly what LeetCode 3310, Remove Methods From Project, is testing.

This problem shows up in interviews because it's really two classic ideas stitched together: graph reachability (find everything downstream of a starting node) and a safety check on that result (make sure nothing outside the group depends on it). Individually, both are things most engineers can do in their sleep. Together, in a 20-30 minute interview window, they trip people up because it's tempting to stop after step one and declare victory.

Once you've worked through this problem, you'll recognize the pattern immediately in real systems: removing a broken microservice and its transitive dependents, deleting a package and everything that imports it, or pruning dead code in a build graph. The mechanics are identical — flood outward from the source, then check the perimeter before you commit to deleting anything.

Problem Overview

You're given `n` methods, numbered `0` to `n - 1`, and an integer `k` identifying a method that has a bug. You're also given `invocations`, a list of pairs `[a, b]` where each pair means "method `a` calls method `b`."

Because method `k` is buggy, every method it calls — directly or transitively — is considered suspicious too, since a broken method can corrupt anything downstream of it. Your job is to return the list of methods that remain safe to keep after removing `k` and its suspicious dependents.

There's one catch: you can only actually remove the suspicious group if nothing outside that group calls into it. If some healthy method still depends on a method inside the suspicious set, removing the group would break that healthy method, so in that case you remove nothing at all and return every method unchanged.

Example

Suppose $n = 5$ and $k = 1$, with:

```
invocations = [[0, 1], [0, 2], [1, 2], [3, 4]]
```

Reading each pair as caller $\rightarrow$ callee: method 0 calls 1 and 2, method 1 calls 2, and method 3 calls 4 (an unrelated pair off on its own).

Starting from $k = 1$: method 1 calls method 2, so $\{1, 2\}$ is suspicious. Method 0 is not suspicious itself, but it calls method 1, which is suspicious — that's an incoming edge from outside the group. Because of that edge, we can't remove anything. The answer is $[0, 1, 2, 3, 4]$, every method, untouched.

Change the starting point to $k = 3$. Method 3 only calls method 4, so $\{3, 4\}$ is suspicious. No other method calls into 3 or 4. Nothing blocks the removal, so the answer is $[0, 1, 2]$ — everything except the suspicious group.

The difference between these two examples is the entire problem: reachability finds the candidates, but the incoming-edge check decides whether you're allowed to act on them.

Intuition

Start by separating the problem into two questions, because trying to solve them at once is where confusion creeps in.

Question one: what's suspicious? This is pure forward reachability. Starting at k , anything you can reach by following "calls" edges is suspicious — k itself, plus everything it calls, plus everything those calls call, and so on. This is a textbook flood fill: seed a frontier with k , expand outward one hop at a time, and stop when there's nothing new left to visit.

Question two: is it safe to remove? This is where the problem stops being a pure traversal exercise. A method being suspicious doesn't mean it's isolated — other, perfectly healthy methods might still call into the suspicious group from outside. If you deleted the group anyway, those healthy methods would be left calling into nothing. So before removing anything, you scan every edge in the graph and ask: does this edge start outside the suspicious set and end inside it? If even one edge does, the whole group is off-limits — not just parts of it, all of it — because you can't cleanly separate "used" from "buggy" once there's a live dependency crossing the boundary.

The key mental shift is realizing these are two separate passes over the graph, not one combined check. First you flood outward to find who's suspicious. Only after that set is finalized do you look at the graph a second time, checking every edge for the reverse case — someone healthy pointing in.

Brute Force Solution

Idea: For every method, run a full traversal to determine whether it's reachable from `k`. Then, for that same suspicious set, iterate over every invocation pair and check if it crosses into the group from outside.

Algorithm: 1. Build an adjacency list from `invocations`. 2. Run DFS/BFS from `k` to compute the suspicious set — this part isn't actually "brute force" different from the optimal approach, since flood fill is already the right tool. 3. For the incoming-edge check, loop over every `[a, b]` pair and test membership in the suspicious set with a linear scan through a list instead of a set.

Advantages: Conceptually simple, easy to reason about correctness.

Disadvantages: If membership checks use lists instead of hash sets, each check costs $O(\text{size of suspicious set})$ instead of $O(1)$, pushing the total complexity toward $O(n \times e)$ in the worst case — far too slow once `n` and the edge count approach 100,000, which the constraints allow.

Complexity: Time $O(n \times e)$ with naive membership checks, Space $O(n + e)$ for the graph.

The fix is almost free: swap the list for a set. That single change turns this "brute force" version into the optimal one, which is why this problem doesn't really have a meaningfully different slow solution — the brute force mistake is a data structure choice, not an algorithmic one.

Optimal Solution

The optimal solution is the same two-phase idea, done with $O(1)$ membership checks throughout.

Phase 1 — Build the graph. For every `[a, b]` in `invocations`, append `b` to `graph[a]`'s call list. This gives you, for any method, the exact list of methods it calls directly.

Phase 2 — Flood fill from k. Use a set called `suspicious` and a stack, both seeded with `k`. Pop a method off the stack, look at everything it calls via `graph`, and for each callee not already in `suspicious`, add it to the set and push it onto the stack. Repeat until the stack is empty. When it's empty, `suspicious` contains exactly the connected group reachable from `k`.

Step	Stack	Suspicious set
Start	<code>[k]</code>	<code>{k}</code>
Pop <code>k</code> , push its callees	<code>[callees of k]</code>	<code>{k, callees of k}</code>
Pop each callee, push new callees ...		grows until no new nodes found
Stack empty	<code>[]</code>	final suspicious set

Phase 3 — Check incoming edges. Iterate over every `[a, b]` in `invocations` once. If `b` is in `suspicious` and `a` is not, you've found a healthy method calling into the suspicious group — removal is blocked, so return every method from `0` to `n - 1` unchanged.

Phase 4 — Build the result. If no blocking edge was found, return every method from `0` to `n - 1` that is not in `suspicious`.

Python Code

```
def remove_methods(n: int, k: int, invocations: list[list[int]]) -> list[int]:
    # Build the call graph: method -> list of methods it calls
    graph = [[] for _ in range(n)]
    for caller, callee in invocations:
        graph[caller].append(callee)

    # Phase 1: flood fill from k to find every suspicious method
    suspicious = {k}
    stack = [k]
    while stack:
        method = stack.pop()
        for callee in graph[method]:
            if callee not in suspicious:
                suspicious.add(callee)
                stack.append(callee)

    # Phase 2: block removal if any healthy method calls into the group
    for caller, callee in invocations:
        if callee in suspicious and caller not in suspicious:
            return list(range(n))

    # Phase 3: everything outside the suspicious set survives
    return [method for method in range(n) if method not in suspicious]
```

Code Walkthrough

- `graph = [[] for _ in range(n)]` — one adjacency list slot per method, built once so lookups during the flood fill are $O(1)$.
- The first loop populates `graph` by appending each `callee` to its `caller`'s list — this is just re-expressing the edge list as an adjacency list.
- `suspicious = {k}` and `stack = [k]` — a set for instant membership checks, a stack (not recursion) so a long dependency chain can't blow Python's recursion limit.
- The `while stack` loop is standard iterative DFS: pop a method, examine its direct callees, and push any unseen ones onto the stack after marking them.

- The second loop over `invocations` is the safety check — it's a separate, full pass over every edge, checking specifically for edges that enter the suspicious set from outside it.
- Returning `list(range(n))` on a blocked removal means "nothing changes" — every method survives, including `k`.
- The final list comprehension filters out the suspicious set only when removal was never blocked.

Dry Run

Take `n = 5`, `k = 0`, `invocations = [[0, 1], [0, 2], [1, 2], [3, 4]]` (using the first example's edges but starting from 0 this time).

Build graph: `graph[0] = [1, 2]`, `graph[1] = [2]`, `graph[3] = [4]`, `graph[2] = graph[4] = []`.

Flood fill: `suspicious = {0}`, `stack = [0]` .- Pop `0`. Callees `1, 2` are new → `suspicious = {0, 1, 2}`, `stack = [1, 2]` .- Pop `2`. No callees. `stack = [1]` .- Pop `1`. Callee `2` already in `suspicious`. `stack = []`.

Final `suspicious = {0, 1, 2}`.

Incoming edge check: Scan `[0,1]`, `[0,2]`, `[1,2]` — callers `0` and `1` are both already inside `suspicious`, so no blocking edge. `[3,4]` — callee `4` isn't in `suspicious` at all, irrelevant.

Result: No block found, so return methods not in `{0, 1, 2}` → `[3, 4]`.

Complexity Analysis

Time: $O(n + e)$, where `e` is the number of invocations. Building the graph is $O(e)$. The flood fill visits each node at most once and each edge at most once (a node only enters the stack the first time it's discovered), so that's $O(n + e)$. The incoming-edge check is one pass over all edges, $O(e)$. Building the final list is $O(n)$. Everything sums to $O(n + e)$.

Space: $O(n + e)$ for the adjacency list, the suspicious set, and the stack, all of which are bounded by the number of nodes and edges.

These bounds are tight because you fundamentally cannot solve this problem without looking at every method and every edge at least once — there's no shortcut around examining the full input.

Alternative Solutions

You can write the flood fill recursively instead of with an explicit stack:

```
def dfs(method, graph, suspicious):
    suspicious.add(method)
    for callee in graph[method]:
        if callee not in suspicious:
            dfs(callee, graph, suspicious)
```

It reads slightly cleaner, but with `n` up to 100,000, a long linear chain of calls (`0 → 1 → 2 → ... → 99999`) can exceed Python's default recursion limit and crash. The stack-based version has identical time and space complexity but never hits that ceiling, so it's the safer default here — this is one of the rare cases where the "less elegant" version is unambiguously the right call.

BFS with a queue instead of a stack works identically well; the traversal order doesn't matter for this problem, only that every reachable node gets visited exactly once.

Edge Cases

- **k calls nothing:** `suspicious = {k}` only. If nothing points into `k`, it's removed cleanly by itself.
- **No invocations at all:** The incoming-edge check loop never runs, so nothing can block removal — `k` and anything it (vacuously) calls just gets removed.
- **The graph has a cycle back through k:** If every method ends up reachable from `k` (a fully connected or cyclic graph), `suspicious` ends up being all `n` methods, and the result is an empty list — everything is suspicious, nothing outside exists to point back in.
- **Duplicate invocation pairs:** Duplicates just get added to `graph[caller]` more than once; they don't change correctness, though a defensive implementation could dedupe them for a small constant-factor speedup.
- **k is unreachable from anything but has huge fan-out:** The flood fill still correctly captures the whole downstream set regardless of how large it is; complexity scales with the number of edges touched, not with `n` alone.

Common Mistakes

1. **Skipping the incoming-edge check entirely** and always removing the suspicious set — this passes on graphs where nothing calls back in, then silently fails the moment a healthy caller exists.
2. **Removing only part of the suspicious group** when a blocking edge is found, instead of blocking the removal completely. The rule is all-or-nothing: one bad edge cancels the entire

operation.

3. **Using plain recursion for the traversal** without considering `n` up to 100,000 — a deep call chain will hit Python's recursion limit.
4. **Forgetting that `k` itself is suspicious**, not just what it calls — some implementations start the suspicious set empty and only add `k`'s callees, missing `k`.
5. **Checking membership with a list instead of a set**, turning an $O(n + e)$ solution into something quadratic under the hood without any visible change to the algorithm's structure.

Interview Questions

- How would you extend this to handle multiple simultaneously buggy starting methods?
- What changes if the graph is undirected instead of directed?
- How would you modify this to also report *why* removal was blocked — i.e., return the specific blocking edge?
- Could you solve the incoming-edge check without a second full pass over `invocations`? (You can fold it into the flood fill if you also track reverse edges, but it doesn't improve asymptotic complexity.)
- How does this generalize to strongly connected components in a general dependency graph?

Similar Problems

- **LeetCode 802 (Find Eventual Safe States)** — also about reachability and separating "safe" nodes from a computed unsafe set.
- **LeetCode 207 / 210 (Course Schedule I & II)** — dependency graph traversal with cycle detection, same adjacency-list-plus-DFS backbone.
- **LeetCode 1319 (Number of Operations to Make Network Connected)** — connected component counting, same flood-fill mechanics.
- **LeetCode 947 (Most Stones Removed with Same Row or Column)** — reachability groups determine what can safely be removed, a close conceptual cousin.

Key Takeaways

Reachability alone answers "what's connected to the source," but real deletion problems almost always add a second question: "does anything outside still depend on this group." Solve those as two clean, separate passes — flood fill first, then a perimeter check — rather than trying to merge them into one clever loop. And whenever `n` can get large, default to iterative traversal over recursion; it costs nothing and removes an entire class of crash.

Watch the Video

For the full walkthrough with the graph drawn out live, both the stack and recursive DFS versions compared side by side, and the quick quizzes at the end to test yourself, watch the video here: {LINK}

About the Series

This is part of Fun with Learning Technology, a series that takes the interview problems that trip people up and breaks them down until the pattern actually clicks — not just for this one problem, but for the next ten that look like it.

Call To Action

If this walkthrough helped, the biggest support is a like on the video and a subscribe so future breakdowns land in your feed. For deeper dives and written companions to every problem, subscribe to the Substack. And if something in this explanation could be clearer, or you want a specific problem covered next, drop it in the comments — that feedback shapes what gets made next.

Quick Review

What pattern does 'Remove Methods From Project' signal?

Graph deletion via BFS/DFS reachability — when you see 'remove dependents of a broken node', think graph traversal from a source.

Time complexity for building graph + traversal here?

$O(n + k)$ — n nodes, k edges, each visited/built once via adjacency list.

Space complexity of the solution?

$O(n + k)$ for adjacency list plus $O(n)$ for visited/queue set.

Trickiest edge case in this problem?

Edges pointing to already-removed nodes or cycles — must track visited to avoid re-processing or infinite loops.

Reachability-from-source graph deletion vs standard topological sort?

Here you mark everything reachable from the buggy node for removal, not order all nodes by dependency.

Interview follow-up: what if edges were undirected?

Reachability set could balloon to unrelated methods — you'd need directed edges to isolate only true dependents.

Remove Methods From Project — free Python cheat sheet